

Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación y Diseño Digital

Carrera de Ingeniería de Software



Guía 1: Sockets TCP y UDP

Autores:

Jeremy Alexis Alvarez Parraga.

Jhinson Stalyn Aucatoma Celorio.

Carlos Jose Carpio Mendoza.

Martes 12 de mayo del 2026

Repositorio GitHub

[https://github.com/JhinsonAucatoma/Sistemas_distribuidos/tree/main/
Trabajos_Grupales/Practica_experimental1](https://github.com/JhinsonAucatoma/Sistemas_distribuidos/tree/main/Trabajos_Grupales/Practica_experimental1)

Índice general

1. Desarrollo de la práctica	2
1.1. Implementación del Socket TCP	2
1.1.1. Desarrollo del servidor TCP	2
1.1.2. Comandos implementados	3
1.1.3. Desarrollo del cliente TCP	3
1.1.4. Pruebas realizadas	4
1.2. Implementación del Socket UDP	5
1.2.1. Desarrollo del servidor UDP	5
1.2.2. Desarrollo del cliente UDP	6
1.2.3. Manejo de timeout	7
1.2.4. Pruebas realizadas	7

Capítulo 1

Desarrollo de la práctica

1.1. Implementación del Socket TCP

Para el desarrollo de la práctica se creó un proyecto Java en IntelliJ IDEA utilizando Maven. Dentro de la carpeta `src/main/java` se creó el paquete:

```
1 ec.edu.uteq.distribuidas.tcp
```

En este paquete se desarrollaron las clases `ServidorTCP` y `CienteTCP`, las cuales permiten establecer una comunicación mediante el protocolo TCP.

1.1.1. Desarrollo del servidor TCP

La clase `ServidorTCP` fue creada con el objetivo de esperar conexiones de clientes a través del puerto 9000. Para lograr esto se utilizó la clase `ServerSocket`, la cual permite que el servidor permanezca en ejecución escuchando solicitudes de conexión enviadas por otros equipos o programas.

Además, se implementó un conjunto fijo de hilos para que varios clientes puedan conectarse al mismo tiempo sin interrumpir el funcionamiento del servidor.

```
1 private static final int PUERTO = 9000;  
2 private static final int MAX_HILOS = 10;
```

El siguiente fragmento de código muestra cómo el servidor permanece continuamente esperando nuevas conexiones:

```
1 while (true) {  
2     Socket cliente = servidor.accept();  
3 }
```

La instrucción `accept()` detiene temporalmente la ejecución hasta que un cliente intenta conectarse. Una vez establecida la conexión, el servidor acepta al cliente y continúa su funcionamiento.

Cada vez que un cliente se conecta, el servidor crea un hilo independiente utilizando la clase interna `ManejadorCliente`. Esto permite atender varios clientes simultáneamente sin que uno bloquee a los demás.

1.1.2. Comandos implementados

Para probar la comunicación entre cliente y servidor, se implementaron tres comandos principales:

- **HORA**: devuelve la hora actual del servidor.
- **ECO**: retorna el mismo mensaje enviado por el cliente.
- **SALIR**: finaliza la conexión entre cliente y servidor.

Para identificar cada comando se utilizaron estructuras condicionales que verifican el texto recibido desde el cliente.

```
1 if (linea.equalsIgnoreCase("HORA")) {  
2  
3 } else if (linea.toUpperCase().startsWith("ECO")) {  
4  
5 } else if (linea.equalsIgnoreCase("SALIR")) {  
6  
7 }
```

Dependiendo del comando ingresado, el servidor genera y envía la respuesta correspondiente.

1.1.3. Desarrollo del cliente TCP

La clase `ClienteTCP` fue creada para conectarse al servidor utilizando la dirección local `localhost` y el puerto `9000`.

```
1 private static final String HOST = "localhost";  
2 private static final int PUERTO = 9000;
```

El cliente permite al usuario ingresar comandos desde el teclado mediante la clase `Scanner`. Posteriormente los mensajes son enviados al servidor y el cliente espera la respuesta correspondiente.

```
1 salida.println(comando);  
2 String respuesta = entrada.readLine();
```

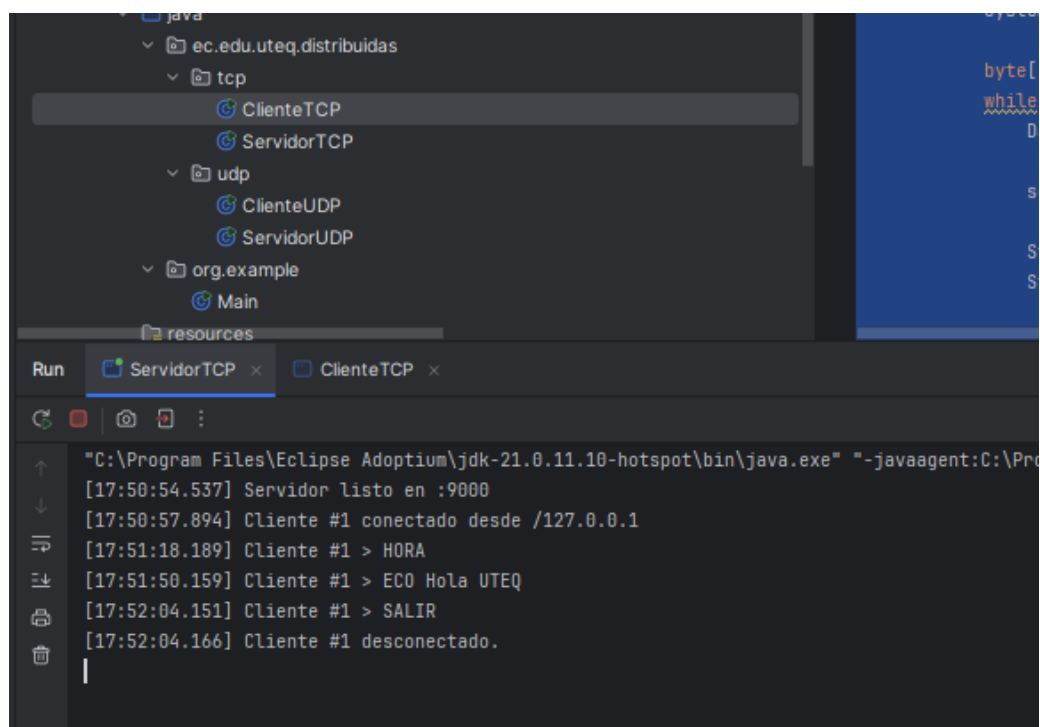
1.1.4. Pruebas realizadas

Para la verificación de la práctica se probaron los diferentes comandos implementados:

- Consulta de hora mediante el comando HORA.
- Envío de mensajes usando ECO.
- Finalización de conexión con SALIR.

El servidor mostró correctamente las conexiones y desconexiones de clientes, así como los mensajes recibidos en consola.

A continuación, se presentan la ejecución del servidor y cliente TCP.



```
"C:\Program Files\Eclipse Adoptium\jdk-21.0.11-hotspot\bin\java.exe" "-javaagent:C:\Pro
[17:50:54.537] Servidor listo en :9000
[17:50:57.894] Cliente #1 conectado desde /127.0.0.1
[17:51:18.189] Cliente #1 > HORA
[17:51:50.159] Cliente #1 > ECO Hola UTEQ
[17:52:04.151] Cliente #1 > SALIR
[17:52:04.166] Cliente #1 desconectado.
```

Figura 1.1: Ejecución del servidor TCP.

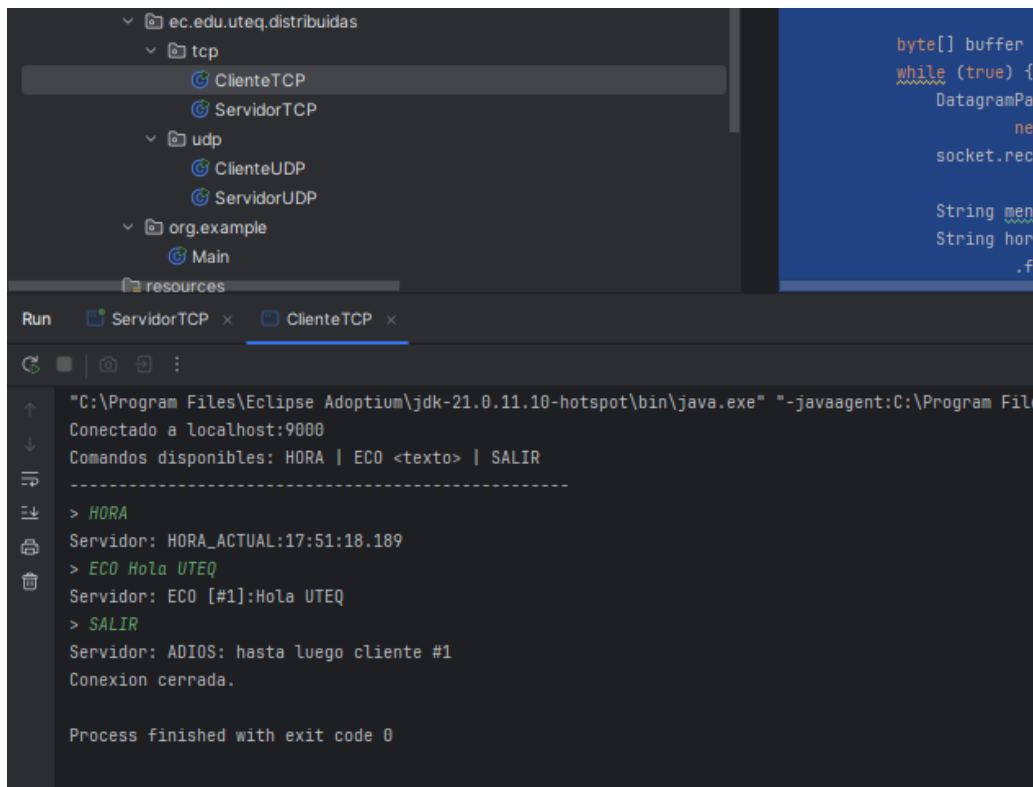


Figura 1.2: Prueba de comunicación entre cliente y servidor TCP.

1.2. Implementación del Socket UDP

Para la implementación del protocolo UDP se creó el paquete:

```
1 ec.edu.uteq.distribuidas.udp
```

Dentro de este paquete se desarrollaron las clases `ServidorUDP` y `ClienteUDP`, las cuales permiten la comunicación mediante el protocolo UDP.

A diferencia de TCP, UDP no establece una conexión permanente entre cliente y servidor. En este protocolo la información se envía mediante paquetes llamados datagramas, lo que permite una comunicación más rápida, aunque con menor control y confiabilidad.

1.2.1. Desarrollo del servidor UDP

La clase `ServidorUDP` fue creada para recibir mensajes enviados por los clientes a través del puerto 9001 utilizando la clase `DatagramSocket`.

```
1 private static final int PUERTO = 9001;
2 private static final int TAM_BUF = 1024;
```

El servidor permanece en ejecución constante esperando mensajes enviados por los clientes.

```
1 while (true) {
```

```

2   DatagramPacket paquete =
3       new DatagramPacket(buffer, buffer.length);
4
5   socket.receive(paquete);
6 }

```

Cada vez que se recibe un datagrama, el servidor obtiene el mensaje enviado, registra la dirección IP y el puerto del cliente, y luego genera una respuesta de confirmación.

```

1 String respuesta = "ACK : " + mensaje + " @" + hora;

```

La respuesta es enviada nuevamente al cliente utilizando el mismo socket y la dirección obtenida desde el paquete recibido.

```

1 DatagramPacket respPkt = new DatagramPacket(
2     resp,
3     resp.length,
4     paquete.getAddress(),
5     paquete.getPort()
6 );
7
8 socket.send(respPkt);

```

1.2.2. Desarrollo del cliente UDP

La clase ClienteUDP fue desarrollada para enviar mensajes al servidor utilizando datagramas UDP mediante la dirección local localhost y el puerto 9001.

```

1 private static final String HOST = "localhost";
2 private static final int PUERTO = 9001;

```

El cliente permite ingresar mensajes desde el teclado mediante la clase Scanner. Cada mensaje es convertido en bytes para poder ser enviado a través de la red.

```

1 byte[] datos = linea.getBytes();
2
3 DatagramPacket pkt = new DatagramPacket(
4     datos,
5     datos.length,
6     servidor,
7     PUERTO
8 );
9
10 socket.send(pkt);

```

Después del envío, el cliente espera la respuesta enviada por el servidor.

```
1 socket.receive(resp);
```

1.2.3. Manejo de timeout

Debido a que UDP no garantiza la entrega de mensajes, se implementó un tiempo máximo de espera para evitar que el cliente permanezca esperando indefinidamente una respuesta del servidor, en este caso, se estableció un tiempo máximo de 3 segundos.

```
1 socket.setSoTimeout(TIMEOUT);
```

Si el tiempo de espera supera los 3 segundos y no se recibe respuesta, el sistema captura una excepción y muestra un mensaje indicando que ocurrió un timeout.

```
1 catch (SocketTimeoutException e) {  
2     System.out.println(  
3         "Timeout: no se recibió respuesta en "  
4         + TIMEOUT + "ms"  
5     );  
6 }
```

1.2.4. Pruebas realizadas

Para comprobar el correcto funcionamiento del protocolo UDP se realizaron pruebas enviando diferentes mensajes desde el cliente hacia el servidor.

El servidor mostró en consola la recepción de los mensajes junto con la dirección IP y el puerto del cliente conectado. De igual manera, el cliente recibió correctamente los mensajes de confirmación enviados por el servidor.

A continuación, se presentan evidencias de funcionamiento del servidor y cliente UDP.

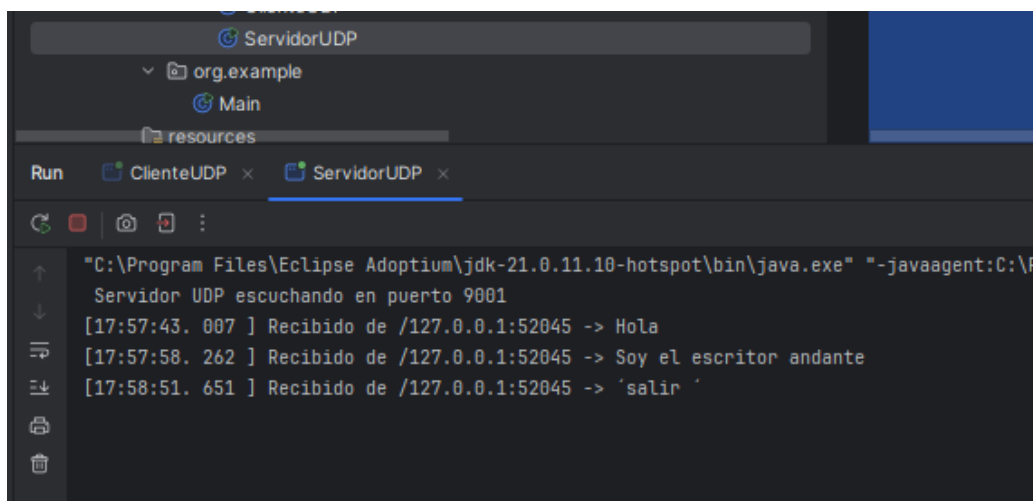
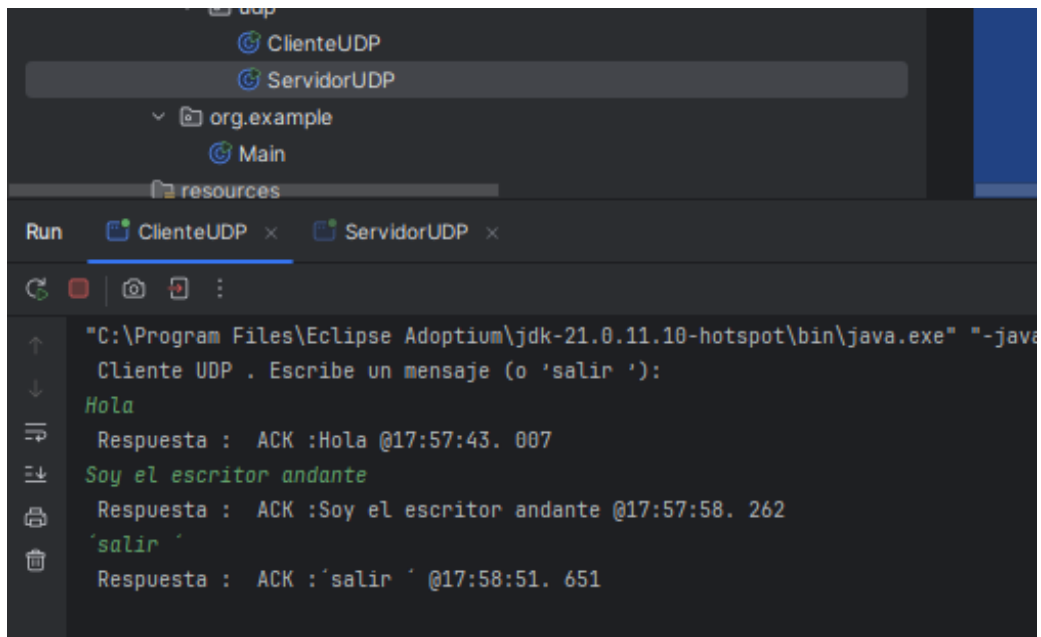


Figura 1.3: Ejecución del servidor UDP.



```
"C:\Program Files\Eclipse Adoptium\jdk-21.0.11-hotspot\bin\java.exe" "-java
Cliente UDP . Escribe un mensaje (o 'salir '):
Hola
Respuesta : ACK :Hola @17:57:43. 007
Soy el escritor andante
Respuesta : ACK :Soy el escritor andante @17:57:58. 262
'salir '
Respuesta : ACK : 'salir ' @17:58:51. 651
```

Figura 1.4: Prueba de comunicación mediante datagramas UDP.